Aula 09 - Segurança da Informação e Boas Práticas no Consumo de APIs

Descrição: Esta aula introduz os conceitos fundamentais de segurança necessários ao projetar e codificar interfaces que se comunicam com APIs. O foco será implementar estratégias *client-side* que respeitem as políticas do servidor, compreendendo que a segurança começa e termina com a proteção dos dados manipulados pela aplicação.

Objetivos da Aula

- Aplicar boas práticas de segurança durante a integração de serviços web.
- Compreender e proteger as operações CRUD (Create, Read, Update, Delete) no front-end.
- Implementar restrições básicas de segurança na interface client-side.
- Desenvolver a disciplina da programação defensiva e autogestão.

Os Dados São o Tesouro e os Pilares da Segurança

O que acontece quando o Instagram de alguém é hackeado?

A Segurança da Informação e seus Pilares no Front-End: O foco principal de qualquer estratégia de segurança não está apenas em se proteger de hackers ou proteger a rede, mas sim em proteger os dados gerados, acessados e manipulados pelas operações CRUD da aplicação. Os dados são o recurso mais importante que qualquer negócio possui, sendo o verdadeiro foco da segurança, muito além da simples preocupação com hackers, aplicações ou infraestrutura de rede. A interação de uma aplicação com os dados baseia-se em quatro operações fundamentais conhecidas pelo acrônimo CRUD: Create (*Criar*), Read (*Ler*), Update (*Atualizar*) e Delete (*Apagar*). Manter esses dados protegidos exige o entendimento pleno das ameaças no ambiente web que envolvem essas operações de interface.

Confidencialidade e Integridade: Garantir que os dados não sejam roubados, corrompidos, modificados indevidamente ou apagados por entidades não autorizadas, prevenindo injeções de código malicioso. Para garantir a Confidencialidade e Integridade, é essencial evitar que os dados sejam modificados, corrompidos, roubados ou apagados, cenários críticos que podem causar perdas sérias e até mesmo a extinção de um negócio. A aplicação front-end deve ter controle estrito sobre como interage com as informações, gerenciando os dados sem danificá-los ou causar vazamentos.

Disponibilidade: Compreender a inter-relação entre velocidade, confiabilidade e segurança. Uma aplicação segura demais pode se tornar inutilizável ou lenta, enquanto uma aplicação sem proteções pode falhar sob carga de ataques (como negação de serviço - DoS), tornando-se indisponível. O pilar da Disponibilidade está intimamente ligado ao *desempenho* e à *usabilidade* do sistema. Existe uma inter-relação direta entre a velocidade, a confiabilidade e a segurança de uma aplicação. Não é possível dar destaque excessivo a apenas um desses aspectos sem que a aplicação seja prejudicada de alguma forma (por exemplo, excesso de verificações tornando a interface lenta ou inutilizável). Aplicações devidamente balanceadas têm bom desempenho e tratam os dados de forma confiável e eficiente, garantindo a segurança sem representar um peso excessivo ou frustração para o usuário final.

Ocultação de Dados (Data Hiding): Disciplina para fornecer e exibir ao usuário final apenas a quantidade de informações *estritamente necessária* para executar uma tarefa específica na interface, reduzindo a superfície de exposição. Muitas aplicações falham ao deixar informações e recursos excessivos disponíveis na interface, o que aumenta a superfície de ataque e oferece mais oportunidades para um invasor ter acesso ao sistema. Livrar-se de recursos desnecessários ou transferi-los para fora do acesso direto do cliente reduz drasticamente essa superfície, tornando a aplicação client-side muito mais segura.

Atividade: Análise de uma interface real. Os alunos devem debater quais informações estão ocultas e quais estão visíveis para o usuário comum.

Boas Práticas: Do Controle de Acesso à Criptografia

Controle de Acesso e Autenticação: No desenvolvimento de interfaces seguras, é fundamental ter certeza sobre com quem a aplicação está interagindo. Deve-se evitar o uso de contas de convidado genéricas para qualquer finalidade, atribuindo sempre uma conta específica a cada usuário. Mesmo quando a identidade do usuário é conhecida, é importante fornecer apenas o nível de autorização estritamente necessário para que ele execute uma determinada tarefa. Qualquer medida que limite o que o usuário está autorizado a fazer contribui significativamente para manter um ambiente seguro. Outro cuidado essencial é a eliminação da autenticação fixa: os desenvolvedores muitas vezes deixam credenciais (*hardcoded*) em arquivos de inicialização para testes, sendo imprescindível remover esses dados do código-fonte e utilizar áreas de armazenamento centralizadas e fora da raiz da aplicação web.

Criptografia e Canais Seguros (TLS/SSL):
Todas as transmissões de dados entre o *client-side* e o servidor devem ocorrer através de conexões seguras, evitando ataques de interceptação (*man-in-the-middle*). No front-end, dados extremamente sensíveis (como senhas, tokens vitais ou dados bancários) nunca devem ser armazenados em disco (armazenamento local) de forma não criptografada, pois ficam acessíveis a usuários do mesmo computador ou a spywares.
Para proteger os dados em trânsito, a criptografia é indispensável nas transmissões entre dois endpoints, evitando que hackers interceptem ou ouçam a comunicação. A implementação desse canal de comunicação seguro e resistente a bisbilhoteiros ocorre através da tecnologia Secure Sockets Layer (SSL) e de seu sucessor mais moderno, o Transport Layer Security (TLS). Recomenda-se sempre a adoção de padrões modernos, como o AES (Advanced Encryption Standard), e a utilização de chaves mais longas para dificultar a quebra da criptografia. No escopo do front-end, é vital compreender que os recursos de armazenamento de dados no lado do cliente (como o *localStorage*) salvam as informações no disco rígido do usuário na forma não criptografada. Por isso, dados armazenados localmente ficam vulneráveis a malwares, spywares e a usuários curiosos que compartilham o mesmo computador. Consequentemente, nenhuma forma de armazenamento no *client-side* deve ser usada para guardar senhas, dados bancários ou outras informações sigilosas.

Auditoria e Monitoramento de Respostas: Ao consumir serviços (especialmente APIs externas), a aplicação deve estar preparada para auditar retornos inesperados. Entradas e respostas errôneas muitas vezes não ocorrem por acaso e podem sinalizar uma tentativa de ataque em andamento.
A auditoria contínua e o monitoramento das respostas do servidor complementam a arquitetura de segurança da aplicação. É essencial criar mecanismos no código para monitorar as respostas recebidas pelas requisições

às APIs, prestando atenção especial às respostas decorrentes de entradas errôneas. Ao documentar e analisar como uma API reage a anomalias, o desenvolvedor front-end consegue estabelecer um indicador de segurança, para detectar problemas antecipadamente. Na maioria dos casos, requisições errôneas frequentes não ocorrem por acaso e podem oferecer pistas claras sobre a origem de uma vulnerabilidade ou de uma tentativa ativa de ataque.

Atividade Prática - "Caça ao Erro": Entregar aos alunos um trecho de código JavaScript fictício. Eles devem atuar como auditores para encontrar falhas.

Políticas do Navegador: O Guardião da Interface

Política de Mesma Origem (Same-Origin Policy): A política da mesma origem (*same-origin policy*) é uma restrição de segurança fundamental que delimita o conteúdo web com o qual o código JavaScript pode interagir. O navegador de Internet exige que os scripts sejam provenientes do mesmo domínio por razões de segurança, garantindo a proteção dos dados. Especificamente, esta política define que um script apenas pode ler as propriedades de documentos que tenham a mesma origem e que um site só poderá fazer solicitações em *background* (como requisições assíncronas) para serviços que estejam no exato mesmo domínio. Esta diretriz é crucial para proteger os utilizadores e evitar que código mal-intencionado leia ou altere os seus dados, o que poderia comprometer a privacidade ou resultar em fraudes.

CORS (Cross-Origin Resource Sharing): Para flexibilizar esta restrição de forma segura e permitir a comunicação com APIs externas, utiliza-se o CORS (*Cross-Origin Resource Sharing*). O CORS permite que sites partilhem os seus dados de uma maneira que possibilita aos utilizadores incluí-los em interações diretamente no lado do cliente (*client-side*).
Para autorizar esta partilha, o servidor tem de responder com cabeçalhos HTTP específicos, tais como:

- Access-Control-Allow-Origin : Permite o acesso exclusivamente para uma origem específica. É a prática mais segura para aplicações em produção;
- Access-Control-Allow-Methods: Ele especifica quais métodos HTTP (como GET, POST, PUT, DELETE, etc.) são permitidos ao acessar um recurso a partir de uma origem diferente daquela que o serviu;
- Access-Control-Allow-Credentials: Informa ao navegador se ele deve ou não expor a resposta de uma requisição de **origem cruzada** (cross-origin) ao código JavaScript do frontend quando credenciais são incluídas.

Estes cabeçalhos definem quais os domínios de origem que estão autorizados a consumir os dados, se o envio de credenciais é permitido e quais os métodos HTTP são aceitos pelo serviço.
Além de viabilizar a comunicação entre domínios distintos, o CORS atua como uma robusta medida de defesa. Pode ser implementado para impedir que sites não autorizados executem tarefas prejudiciais contra uma API, bloqueando tentativas de ataques como o CSRF (*Cross-Site Request Forgery*), onde um invasor tenta explorar a confiança estabelecida num sistema (como um site bancário). Ao validar rigidamente as origens através destes cabeçalhos, o servidor assegura que a interface *client-side* interage exclusivamente num ambiente controlado e aprovado.

Dinâmica - "O Porteiro do Servidor": Jogo de simulação onde os alunos representam o Navegador, o Cliente e o Servidor. Eles devem aprovar ou bloquear requisições imaginárias com base nos "crachás" (cabeçalhos CORS) apresentados, fundamentando suas decisões.

Autogestão e Disciplina: A Programação Defensiva

Desconfiança por Padrão: Ao escrever uma aplicação, a disciplina fundamental do desenvolvedor deve ser assumir ***sempre*** que as entradas e saídas estão erradas, que o usuário tratará tudo de forma incorreta e que existe um invasor espiando as ações do sistema. Para evitar os problemas predominantes de segurança, os desenvolvedores precisam desenvolver um "verdadeiro senso de desconfiança", adotando a premissa absoluta de que qualquer dado que não tenha sido criado pessoalmente por eles não é confiável.

Validação Rigorosa: A validação no lado do cliente serve para a proteção do usuário e deve ser utilizada para mantê-lo satisfeito e produtivo. Tudo o que for feito para reduzir a frustração do usuário ao interagir com o sistema também reduzirá os custos de suporte da aplicação e mitigará falhas que geram preocupações com segurança. É imperativo aplicar um processo de sanitização rigoroso, que consiste em remover qualquer script, caractere estranho ou outras informações que não façam parte da resposta esperada a uma solicitação (por exemplo, bloquear letras em campos estritamente numéricos). Além da sanitização da entrada, garantir uma saída que codifique adequadamente os dados fornecidos pelo usuário ajuda a reduzir o potencial para ataques de *Cross-site Scripting* (XSS).

Sandbox e Resiliência: Todo código ou recurso de terceiros carrega riscos inerentes; executar JavaScript fora de um ambiente de *sandbox* (caixa de areia) abre margem para que hackers executem truques sujos na aplicação. Colocar o consumo da API em uma *sandbox* ou em um ambiente virtual controlado possibilita controlar precisamente as ações que a API pode executar, os recursos a que ela pode acessar e, principalmente, como a aplicação interage com ela. O uso de uma zona de segurança de API permite não apenas restringir entradas indesejadas que poderiam causar efeitos desastrosos, mas também testar o código da interface sob diversas condições anômalas — como o envio proposital de dados inválidos ou simulações de sobrecarga —, garantindo que a resiliência do sistema funcione conforme o esperado.